MID-SEM QUESTION BANK (Unit-I + Unit-II ONLY)

How to use: Part 1 has **only questions** – attempt them on paper first. Part 2 has **model answers** with the same numbers. Tags: 📝 = question is based on a topic emphasized in your BPIT assignment (not copied) · 📘 = syllabus topic · ➕ = additional/exam-oriented. *These are practice questions written from the syllabus; not leaked or guaranteed questions.*

PART 1 — QUESTIONS

A. 1-MARK / VERY SHORT (30)

A1. Define a distributed system. A2. Name the three core characteristics of a distributed system. A3. What is transparency? (name any two types) A4. Give one example each of client–server and peer-to-peer systems. A5. What is grid computing? A6. Cloud computing follows which billing model? A7. What is middleware? A8. Name the two categories of system models. A9. What is latency? A10. Expand TCP and UDP. Which is connectionless? A11. What is an internetwork? A12. Name two IPC mechanisms. A13. What is marshalling? A14. What is a remote interface? A15. Which Java interface must every remote interface extend? A16. What is the default port of the RMI registry? A17. What is the role of a stub? A18. What does RPC stand for? A19. What is a notification? A20. What is an omission failure? A21. What is a Byzantine failure? A22. Define clock skew. A23. Define clock drift. A24. State the happened-before rules (names only). A25. State the Lamport rule for receiving a message. A26. What is a critical section? A27. Name two election algorithms. A28. What does the Bully algorithm elect? A29. What are the three conditions of consensus? A30. What is a consistent cut?

B. 2-MARK QUESTIONS (20)

B1. 📝 Define a distributed system and list four characteristics. B2. 📝 Differentiate between client–server and peer-to-peer in two points. B3. State any four advantages of distributed systems. B4. Why do we need system models? B5. 📝 Differentiate Architectural and Fundamental models (two points). B6. Write short note on synchronous and asynchronous distributed systems. B7. 📝 Why is internetworking needed? Name two devices used. B8. Distinguish TCP and UDP (two points). B9. What is distributed shared memory? B10. 📝 What are remote objects and remote object references? B11. What is the role of the RMI registry? B12. State the three invocation semantics. B13. What are events and notifications? Give an example. B14. 📝 What is meant by "no global clock"? Why is it a problem? B15. 📝 Explain process state and event. B16. Differentiate between Lamport and vector clocks (two points). B17. 📝 What are clock drift and skew? How are they related? B18. State the requirements of distributed mutual exclusion. B19. Give message complexity of Ricart–Agrawala, central server and Lamport ME algorithms. B20. What are the types of ordering in multicast?

C. 3-MARK QUESTIONS (12)

C1. 📝 Explain the characteristics of distributed systems with examples. C2. 📝 Explain client–server architecture with a diagram, advantages and disadvantages. C3. Explain the interaction

model with latency, bandwidth and jitter. C4. Explain omission, arbitrary and timing failures. C5. 📝 Explain message passing as an IPC mechanism. C6. 📝 What is a stub and a skeleton? How does a remote call proceed? C7. Explain RPC with a neat diagram. C8. 📝 Explain the happened-before relation with an example. C9. 📝 Explain Cristian's algorithm with example. C10. 📝 What is global state? List its components. C11. Explain publish–subscribe model. C12. Explain reliable multicast properties.

D. 5-MARK QUESTIONS (12)

D1. 📝 Compare client–server, peer-to-peer, grid and cloud computing. D2. 📝 Explain architectural models of distributed systems with diagrams. D3. 📝 Explain networking and internetworking and the role of protocols in distributed applications. D4. 📝 Compare message passing and shared memory. D5. 📝 Explain distributed objects and RMI: remote object, remote interface, stub/proxy and invocation steps. D6. Compare RPC and RMI. D7. 📝 Explain Lamport logical clock with rules and example. D8. 📝 Explain need of physical clock synchronisation; discuss drift, skew, delay. D9. 📝 Explain Chandy–Lamport snapshot algorithm. D10. 📝 Explain Bully algorithm with example. D11. 📝 Explain Ricart–Agrawala algorithm. D12. 📝 Explain consensus and its conditions with an example.

E. 7 / 8 / 10-MARK LONG ANSWERS (8)

E1. 📝 Explain system models: need, architectural models and fundamental models with diagrams. E2. 📝 Design a simple Java RMI application (calculator): architecture, remote interface, server, client, registry, execution steps, code and output. E3. 📝 Explain Interprocess communication in detail; compare message passing and shared memory. E4. 📝 Explain global states, consistent cut and the distributed snapshot algorithm with a P1–P2–P3 example. E5. 📝 Explain distributed mutual exclusion: requirements and four algorithms with message complexity. E6. 📝 Explain election algorithms: Bully and Ring, with examples and comparison. E7. 📝 Explain consensus in distributed systems: requirements, process failures, communication delays, Byzantine generals, related problems. E8. Explain Synchronization: physical clock algorithms (Cristian, Berkeley, NTP) and logical clocks.

F. NUMERICAL / PROBLEM-BASED (8)

F1. 📝 P1: events a, send m1→P2 (b), receive m3 (c). P2: d, receive m1 (e), send m2→P3 (f). P3: g, receive m2 (h), send m3→P1 (i). Assign Lamport timestamps. F2. A process has clock 14 and receives a message timestamped 20; another has clock 30 and receives timestamp 25. Find new clocks. F3. Using Cristian's algorithm: T0 = 12:00:00.000, T1 = 12:00:00.600, server time = 12:00:10.200. Find client's new time and the accuracy bound. F4. 📝 Berkeley: master 2:00, slaves 1:50, 2:05, 2:20 (slave times as clock readings; master included in average). Find each adjustment. F5. Clock drift rate $\rho = 2\times10^{-6}$. Skew allowed 2 ms. How often to resynchronise? How much does a clock drift in one day? F6. Ricart–Agrawala with N = 8: messages per CS entry? Lamport ME? Central server? F7. Bully algorithm: processes P1–P6, P6 fails, P3 detects. List messages sent and the winner. Count messages. F8. Ring election: ring P1→P4→P2→P6→P3→P1 (order as arranged), IDs = numbers, coordinator failed, P2 starts. Show messages until winner.

G. DIAGRAM-BASED (8)

G1. Draw and explain the RMI architecture (proxy, dispatcher, skeleton, communication module). G2. 📝 Draw the software-layer/middleware architecture of a distributed system. G3. Draw the message sequence for an RPC call. G4. 📝 Draw a timeline showing happened-before and concurrent events for 3 processes. G5. 📝 Draw a consistent and an inconsistent cut. G6. 📝 Draw message flow of Chandy–Lamport snapshot for P1, P2, P3. G7. 📝 Draw message flow of Bully election with 5 processes. G8. Draw the hold-back queue for ordered multicast.

H. COMPARE / DIFFERENTIATE (10)

H1. 📝 Client–server vs peer-to-peer. H2. 📝 Message passing vs shared memory. H3. 📝 RMI vs RPC. H4. Synchronous vs asynchronous DS. H5. 📝 Logical vs physical time. H6. Lamport vs vector clocks. H7. 📝 Bully vs Ring algorithm. H8. Cristian vs Berkeley. H9. Token-based vs non-token-based mutual exclusion. H10. Crash vs Byzantine failures.

I. CASE-STUDY QUESTIONS (5)

I1. 📝 A college wants a Student Information Service that lets any lab computer call a method to fetch a student's record from a central server. Design it using Java RMI. I2. 📝 Three bank branch servers P1, P2, P3 hold balances 500, 300, 200. Design a method to record the total consistently while transfers are happening. I3. 📝 Five replicated database servers; one may crash. How do they agree to commit or abort? Discuss challenges. I4. A file server is shared by 6 clients that all update the same file. Choose and justify a mutual-exclusion scheme. I5. In a cluster of 7 servers the coordinator crashes. Show how a new coordinator is chosen with the Bully algorithm and with the ring algorithm.

PART 2 — MODEL ANSWERS

A. 1-MARK ANSWERS

A1. A system whose components on networked computers communicate and coordinate **only by passing messages**. A2. Concurrency, no global clock, independent failures. A3. Hiding the distribution of components from users; e.g., access transparency, location transparency (also replication, failure, mobility, concurrency, performance, scaling). A4. Client–server: web browser–web server. P2P: BitTorrent. A5. Pooling geographically distributed heterogeneous resources of multiple organisations into a virtual supercomputer. A6. Pay-per-use (on-demand, metered). A7. Software layer between OS and applications masking heterogeneity (e.g., RMI, CORBA). A8. Architectural models and fundamental models. A9. Delay between start of transmission at sender and start of receipt at receiver. A10. Transmission Control Protocol, User Datagram Protocol; UDP is connectionless. A11. A network of networks connected by routers acting as a single logical network. A12. Message passing and shared memory (sockets, message queues, DSM). A13. Converting data structures into a flat byte sequence for transmission (and reverse = unmarshalling). A14. Specification of the methods of a remote object that clients may invoke remotely. A15. `java.rmi.Remote`. A16. 1099. A17. Client-side proxy that marshals the call and forwards it to the server, hiding remoteness. A18. Remote Procedure Call. A19. A message describing an event, sent to interested subscribers. A20. A

process or channel fails to perform an action (crash or message loss). A21. Arbitrary behaviour: wrong values, contradictory messages, corruption. A22. Difference between readings of two clocks at the same instant. A23. Rate at which a clock deviates from a perfect reference clock. A24. (1) Same-process order, (2) send → receive, (3) transitivity. A25. `L = max(L, t) + 1`. A26. Code segment that accesses a shared resource and must be executed by one process at a time. A27. Bully and Ring algorithms. A28. The alive process with the highest ID. A29. Termination, agreement, validity (integrity). A30. A cut in which every receive event's send event is also inside the cut (no orphan messages).

B. 2-MARK ANSWERS

B1. Def (A1). Characteristics: concurrency (many users act simultaneously), no global clock (each node has own clock), independent failures (one node can fail alone), resource sharing/heterogeneity/openness/scalability. B2. C-S: fixed roles, central server (SPOF). P2P: symmetric roles, decentralised, no SPOF. B3. Resource sharing, scalability, reliability/availability, better performance, cost-effectiveness, geographical distribution. B4. To make assumptions explicit, identify common design issues, reason about correctness/feasibility of algorithms, and generalise across systems. B5. Architectural: structure/placement of components (client-server, P2P). Fundamental: abstract properties – interaction, failure, security. B6. Synchronous: known bounds on execution time, message delay, drift → timeouts reliable. Asynchronous: no bounds (Internet) → can't distinguish slow from crashed. B7. To connect heterogeneous networks so hosts communicate globally (Internet). Devices: router, gateway (also bridge/switch). B8. TCP – connection-oriented, reliable, ordered. UDP – connectionless, unreliable, faster. B9. DSM gives the illusion of one shared address space over physically separate memories using page faults and message passing. B10. Remote object: object whose methods can be invoked from other processes. Remote object reference: system-wide identifier (address, port, time, object number, interface) used to invoke it. B11. Name service: server binds a name to the stub, client looks it up to obtain a stub (port 1099). B12. Maybe, at-least-once, at-most-once (Java RMI: at-most-once). B13. Event: occurrence of interest (e.g., stock price change). Notification: message about it sent to subscribers via event service. B14. Each computer has its own clock; clocks drift, so events on different machines can't be ordered by their timestamps → need logical clocks/sync algorithms. B15. State = values of a process's variables; event = action (internal/send/receive) that changes state. B16. Lamport: one integer, $a \to b \Rightarrow L(a) < L(b)$ only. Vector: N integers, $a \to b \Leftrightarrow V(a) < V(b)$, detects concurrency. B17. Skew = difference of two clocks at an instant; drift = rate of deviation. Drift makes skew grow with time (skew $\leq 2\rho\Delta t$). B18. Safety (one in CS), liveness (no deadlock/starvation), ordering/fairness (happened-before order). B19. RA = $2(N-1)$; central = 3; Lamport = $3(N-1)$. B20. FIFO, causal, total ordering.

C. 3-MARK ANSWERS (key points)

C1. Definition + concurrency, no global clock, independent failures, heterogeneity, openness, scalability, transparency, fault tolerance. Give example: Google Search / banking (concurrent users, replicas fail independently). C2. Diagram: clients → server. Steps: request, processing, response. Adv: central control, easy security; Disadv: SPOF, bottleneck. Examples: web, email. C3. Interaction model: processes interact by messages; performance = latency (delay), bandwidth (data per sec), jitter (variation in delivery time); transmission time = latency + size/bandwidth; each process has own clock with drift; synchronous vs asynchronous variants. C4. Omission: process crash or channel message loss. Arbitrary/Byzantine: wrong/contradictory

values, worst. Timing: response outside time limit (only synchronous). Masking: replication, checksums. C5. Explicit send/receive; sync (blocking) vs async; sockets (UDP datagram/TCP stream); marshalling; used in DS since no shared memory; advantages: works across network, isolation; disadvantages: slower, programmer handles format/failures. C6. Stub = client proxy (marshals/unmarshals). Skeleton = server side unmarshals, calls servant, marshals reply. Steps: local call on stub → marshal → request via network → dispatcher/skeleton → servant → reply → stub returns result. C7. Diagram (client → client stub → RPC runtime → server stub → procedure) + 8 steps (see notes §10) + IDL/binder. C8. 3 rules; example P1: a→b; b=send m1, P2: receive m1=d ⇒ b→d ⇒ a→d; unrelated events concurrent. C9. Client sends request at T0; server returns T_S; client receives at T1; sets T_S + (T1−T0)/2; accuracy ± (RTT/2 − min); example numbers. C10. Global state = local states of all processes + states of all channels (messages in transit); consistent iff it corresponds to a consistent cut. C11. Publishers generate events; subscribers register; event service filters and notifies asynchronously; loosely coupled; examples: MQTT/Kafka, stock alerts. C12. Integrity (deliver at most once, only if sent), validity (correct sender delivers own message), agreement (if one correct process delivers, all correct do).

D. 5-MARK ANSWERS

D1. Use the 4-way comparison table in `01_Unit1_Notes.md §3.5` (≥8 rows) + one-line definition and an example per system + one diagram each for CS and P2P.

D2. Definition of architectural model → software layers & middleware diagram → client-server → multiple servers → proxy/cache → P2P → mobile code → mobile agents → thin clients (diagram for each of 3–4) → conclusion.

D3. Define network/internetwork → LAN/MAN/WAN → protocols & layers (TCP/IP) → TCP/UDP → routers/gateways → role in DS (message transport, connectivity, heterogeneity handling, QoS: latency/bandwidth) → diagram of internetwork.

D4. Definitions (2 lines each) + diagram of both + comparison table (speed, location, sync, scalability, complexity, security) + advantages/limitations + examples + conclusion "DS uses message passing".

D5. Distributed object def → remote object/interface/reference → local vs remote invocation → architecture diagram (proxy, comm module, remote ref module, dispatcher, skeleton, servant) → 8 steps → semantics → example.

D6. RMI vs RPC table (§10.5): paradigm, parameters, language, stubs, GC, overhead, use.

D7. Definition → LC1–LC3 rules → algorithm → worked example with table → timeline diagram → properties (a→b ⇒ L(a)<L(b), converse false) → limitation → total ordering with process id.

D8. Why: timestamps, logs, Kerberos, files, real-time → skew, drift ($\rho \approx 10^{-6}$, 86.4 ms/day), delay (variable) → sync interval $\delta/2\rho$ → algorithms (Cristian/Berkeley/NTP) → problems.

D9. Purpose → assumptions (FIFO, reliable, marker) → algorithm (marker sending/receiving rules) → example (bank P1,P2,P3 with 450/350/180 and channel ₹20 = 1000) → complexity → uses.

D10. Purpose, assumptions (synchronous, IDs known) → 3 messages → steps → 5-process example → messages best N−2 / worst $O(N^2)$ → pros/cons.

D11. Purpose, states RELEASED/WANTED/HELD → pseudocode → example (5,P1) vs (7,P2) → 2(N−1) messages → pros/cons.

D12. Definition → 3 conditions → example (3 replicas commit/abort) → flooding f+1 rounds → failures/delay issues → FLP.

E. LONG ANSWERS

E1. (10 marks) *Outline to write:* 1. Definition & need (5 points) [1 mark]. 2. **Architectural**: layers/middleware; client-server; multiple servers; proxy & cache; peer processes; mobile code; agents; thin clients; tiers; design requirements [4 marks + diagrams]. 3. **Fundamental**: interaction (latency, bandwidth, jitter, drift, sync/async, event ordering); failure (omission, arbitrary, timing, masking, reliable channel = validity + integrity); security (objects/principals, threats, secure channels, crypto) [4 marks]. 4. Comparison + conclusion [1 mark].

E2. (10 marks) Java RMI calculator — *see full code and output in* `01_Unit1_Notes.md §12.5`. Structure: what is RMI (2 lines) → architecture diagram (client, stub, RMI registry, server, remote object) → components (remote interface extends Remote & throws RemoteException; implementation extends UnicastRemoteObject; server: create object, `createRegistry(1099)`, `rebind`; client: `getRegistry`, `lookup`, call) → 8 execution steps → code (interface, impl, server, client) → sample output (15, 5, 50, 2.0) → advantages/limitations.

E3. (10 marks) IPC — definition; message passing (send/receive, sync/async, sockets UDP/TCP, marshalling, queues); shared memory (concept, semaphores, DSM); diagrams of both; comparison table (≥8 rows); applications; why DS prefers message passing.

E4. (10 marks) Global state & snapshot — global state definition + components (local states + channel states) → cut, consistent vs inconsistent (orphan message) → importance (deadlock/termination detection, checkpointing, debugging) → Chandy–Lamport algorithm (rules for initiator/receiver) → example: P1=500,P2=300,P3=200; m1 ₹50 P1→P2, m2 ₹20 P3→P1; snapshot: 450, 350, 180, C31={20} → total 1000 ✓ → assumptions (FIFO, reliable) → complexity → limitations.

E5. (10 marks) Distributed mutual exclusion — need; requirements ME1–ME3; performance metrics; **central server** (3 msgs), **token ring** (1..N−1), **Ricart–Agrawala** (2(N−1)), **Lamport** (3(N−1)) [+ Maekawa 3√N]; comparison table; failure discussion; conclusion.

E6. (10 marks) Election — need; requirements E1/E2; Bully (assumptions, 3 messages, steps, 5-process example, N−2 / $O(N^2)$); Ring (assumptions, participant flag, steps, example, 2N / 3N−1); comparison table; conclusion.

E7. (10 marks) Consensus — definition, need, three conditions, example; crash consensus flooding f+1 rounds; failures & delays; FLP; Byzantine generals N ≥ 3f+1; interactive consistency; equivalence with total-order multicast; practical protocols (Paxos/Raft).

E8. (10 marks) Synchronization — need for sync; physical clocks (skew, drift, ρ); Cristian, Berkeley, NTP with example numbers; logical clocks: happened-before, Lamport clock rules + example, vector clocks; comparison logical vs physical.

F. NUMERICAL ANSWERS

F1. P1: a=1, b=2 (m1 ts 2). P2: d=1, e=max(1,2)+1=3, f=4 (m2 ts 4). P3: g=1, h=max(1,4)+1=5, i=6 (m3 ts 6). P1: c = max(2,6)+1 = **7**.

F2. 14 & ts 20 → max(14,20)+1 = **21**. 30 & ts 25 → max(30,25)+1 = **31**.

F3. RTT = 0.6 s → new time = 10.200 + 0.3 = **12:00:10.500**. Accuracy ± (RTT/2 − min) = ±0.3 s (if min delay ≈ 0).

F4. Offsets from master: master 0, S1 = −10 min, S2 = +5 min, S3 = +20 min. Average = (0 −10 +5 +20)/4 = **+3.75 min** ⇒ target = 2:03:45. Adjustments: master +3.75 min; S1 = 3.75 − (−10) = **+13.75 min**; S2 = 3.75 − 5 = **−1.25 min**; S3 = 3.75 − 20 = **−16.25 min**. (If a slave is an outlier, e.g., S3, master may ignore it in the average.)

F5. Resync interval ≤ δ/(2ρ) = 0.002 / (2×2×10^{-6}) = 0.002 / 4×10^{-6} = **500 s**. Drift per day = 2×10^{-6} × 86400 = **0.1728 s ≈ 173 ms**.

F6. RA = 2(N−1) = **14**; Lamport = 3(N−1) = **21**; central = **3**.

F7. P3 detects → ELECTION to P4, P5, P6 (3 msgs). P4, P5 reply OK (2). P4 → ELECTION to P5, P6 (2), P5 replies OK (1). P5 → ELECTION to P6 (1), no reply. Also P3's election answered… Count so far: 3+2+2+1+1 = 9; then P5 times out and sends COORDINATOR to P1–P4 (4). Total ≈ **13**. **Winner: P5** (highest alive).

F8. Ring order: P1→P4→P2→P6→P3→P1. P6 assumed failed? (Coordinator failed = P6, skipped.) Alive ring: P1→P4→P2→P3→P1. P2 starts: ELECTION(2) → P3 (3>2: substitute 3, ELECTION(3)) → P1 (1<3: forward 3) → P4 (4>3: substitute 4, ELECTION(4)) → P2 (2<4: forward) → P3 (forward) → P1 (forward) → P4 receives own id 4 → **P4 is coordinator**, sends ELECTED(4) around: P2 → P3 → P1 → P4. Note: the answer depends on which process is treated as failed; the method (substitute larger id, own id returning = winner) is what is examined.

G. DIAGRAM ANSWERS

G1→ `01 §9` RMI architecture diagram with labels. G2→ `01 §5.2(a)`. G3→ `01 §10 / §9 sequence diagram`. G4→ `02 §4 / §5.5`. G5→ `02 §9.2`. G6→ `02 §9.4`. G7→ `02 §13.1`. G8→ `02 §14.3`. *(Always: draw boxes, label each arrow with the message/step name, number the steps, write a one-line caption.)*

H. COMPARISON ANSWERS

H1, H2, H3, H5, H7 → full tables in notes (`01 §3.5`, `§7.3`, `§10.5`, `02 §8`, `§13.3`). **H4** Synchronous vs asynchronous → `03 §6.2`. **H6** → `02 §6`. **H8** → `02 §7.4`. **H9 Token-based vs non-token-based ME** | Token-based (ring, Suzuki–Kasami) | Non-token (Ricart–Agrawala, Lamport) | |---|---| | Holder of unique token may enter | Need permission from other processes using timestamps | | Fewer messages (1..N) | More messages (2(N−1), 3(N−1)) | | Token loss problem | No token; crash of a process blocks | | Safety inherent (single token) | Safety via timestamp ordering | **H10** → `03 §6.16`.

I. CASE-STUDY ANSWERS

I1. Student Information Service (Java RMI) - **Remote interface:** `StudentService extends Remote { String getStudent(int roll) throws RemoteException; }` - **Implementation:** `StudentServiceImpl extends UnicastRemoteObject implements StudentService` with a `HashMap<Integer,String>` of records. - **Server:** create object, `LocateRegistry.createRegistry(1099)`, `rebind("StudentService", obj)`. - **Client (lab computers):** `getRegistry(serverIP,1099)`, `lookup("StudentService")`, `getStudent(101)`. - **Flow:** lookup → stub → marshalled call → server executes → result returned. **Benefits:** location transparency, central data, easy update. Include diagram (client—stub—registry—server).

I2. Consistent total balance – Use Chandy–Lamport with markers: P1 records local balance, sends markers; others record on first marker; record in-transit transfers as channel states; sum of process balances + channel messages = 1000 (example figures 450 + 350 + 180 + 20). Explain FIFO requirement and that application is not stopped.

I3. Five replicas, one may crash – Crash failure: flooding consensus with f+1 = 2 rounds; each broadcasts its vote (commit/abort), re-broadcasts newly learned votes in round 2; apply deterministic rule (e.g., commit only if all votes commit, or majority). Correct processes decide same value: agreement, validity, termination. Challenges: crash mid-multicast (partial delivery), unbounded delays in async systems (FLP), timeout tuning, message overhead $O(N^2)$. If Byzantine: need $N \geq 3f+1 = 4$ ✓ (have 5) and majority voting on values received in second round.

I4. File server with 6 clients – Options: central lock server (3 messages, simple, but SPOF), token ring (fair, but token loss), Ricart–Agrawala (10 messages/entry; no SPOF). Recommendation: **central lock server** (a single existing file server can act as coordinator; minimal messages, natural fit) with election fallback (Bully) if it crashes; justify with message counts (3 vs 10 vs 15 for Lamport).

I5. 7 servers, coordinator P7 crashes – *Bully:* P3 detects → ELECTION to P4,P5,P6,P7; P4,P5,P6 reply OK; each starts election; P6 gets no reply from P7 → declares itself coordinator → COORDINATOR to P1–P5. *Ring:* P3 sends ELECTION(3) → P4 substitutes 4 → P5 → 5 → P6 → 6 → (skip P7) P1 (forward 6) → P2 → P3 → P4 → P5 → P6 sees own id → ELECTED(6) circulates. Both elect P6 (highest alive).